

Resumen del primer video:

## 1. La Realidad del Sector en 2026

Brais comienza desmitificando el miedo a que la IA reemplace a los programadores. Explica que:

- **La IA no te quitará el trabajo**, pero un programador que sepa usar la IA sí podría desplazar a uno que no
- El **30% del código en producción** ya es generado por IA y el 50% de los desarrolladores la usan a diario
- Las empresas ya no buscan "picacódigos" tradicionales, sino **arquitectos y orquestadores** que dominen los fundamentos y sepan guiar a la IA

## 2. Fundamentos Técnicos de la IA

Para usar la IA profesionalmente, es necesario entender cómo funciona "bajo el capó":

- **LLMs y Parámetros:** Explica que son modelos estadísticos que predicen la siguiente palabra basándose en billones de parámetros (conexiones neuronales)
- **Tokens y Costos:** Los tokens son las unidades en las que la IA divide el texto. Es vital entenderlos porque de ellos depende el costo y la eficiencia de las consultas
- **Ventana de Contexto:** Es la "memoria a corto plazo" del modelo. Si se llena, la IA empieza a olvidar partes de la conversación o a alucinar
- **IA Local vs. Cloud:** Muestra herramientas como **LM Studio** y **Ollama** para ejecutar modelos directamente en tu ordenador de forma gratuita y privada, sin depender de la nube

## 3. Ingeniería de Prompts (Prompt Engineering)

Se enseña que un buen prompt debe tener una estructura profesional para evitar resultados mediocres (el concepto de *Garbage In, Garbage Out*):

- **Rol:** Definir quién es la IA (ej. "Actúa como un experto en ciberseguridad").
- **Contexto:** Explicar el proyecto y las tecnologías usadas.
- **Tarea y Restricciones:** Qué debe hacer exactamente y qué límites tiene.

- **Formato de salida:** Cómo quieres recibir el código o la información

#### 4. Herramientas para el Estudio Activo

Brais destaca que la IA ha revolucionado la educación. Presenta **NotebookLM** de Google como una herramienta de **RAG (Generación Aumentada por Recuperación)**

- Permite subir archivos (PDFs, videos de YouTube, apuntes) para que la IA responda **solo basándose en esa fuente de verdad**, evitando alucinaciones
- Puede generar automáticamente cuestionarios, guías de estudio y resúmenes de audio (podcasts) sobre el material proporcionado

#### 5. Spec Driven Development (Desarrollo Guiado por Especificaciones)

Martín introduce esta metodología, que es el núcleo del desarrollo moderno con IA:

- **El cambio de paradigma:** En lugar de pasar horas escribiendo código línea a línea, el programador dedica la mayor parte del tiempo a **definir especificaciones detalladas**
- **Agentes de IA:** A diferencia de un chat normal, un agente puede percibir, decidir y actuar (como ejecutar comandos en la terminal o leer archivos) de forma semiautónoma
- **Flujo de trabajo:** 1. Definir requerimientos (lo que el usuario necesita). 2. Diseñar la solución (arquitectura y diagramas). 3. Ejecutar tareas mediante agentes que generan el código basándose en el diseño previo

#### 6. Herramientas de Desarrollo Mostradas

Durante la clase se realizaron demostraciones de herramientas punteras:

- **Cursor:** Un editor de código diseñado para IA que permite el modo "Agente" y el uso de **Git Worktrees** para que varios agentes trabajen en paralelo sin pisarse el código
- **Cline (Cloud Code):** Un agente que vive en la terminal y puede analizar proyectos completos y ejecutar tests por sí solo
- **Kiro y Codex:** Editores enfocados en el desarrollo agéntico donde el código a veces pasa a un segundo plano frente a la gestión de tareas y especificaciones

En resumen, la lección enseña que para ser un programador profesional en la era de la IA, debes dejar atrás el "vibe coding" (programar por intuición sin entender) y

convertirte en un **líder técnico** que utiliza especificaciones claras para dirigir un "ejército" de agentes de inteligencia artificial.

Resumen del segundo video:

## 1. Evolución de los Editores de Código (IDEs)

Brais explica cómo hemos pasado de simples editores a ecosistemas centrados en la IA (*AI-First*):

- **2022-2023:** Autocompletado simple y chats laterales donde copiabas y pegabas código
- **2024:** Copilotos que interactúan con el archivo abierto.
- **2025-2026:** Agentes autónomos que leen todo el proyecto, ejecutan comandos en la terminal, crean archivos y verifican errores por sí mismos

## 2. Conceptos Clave de Agentes Profesionales

Se profundiza en piezas del puzzle que permiten que la IA sea más inteligente y eficiente:

- **Plan Mode (Modo Plan):** Antes de escribir código, la IA detalla un plan de acción que el programador debe validar. Es ideal para tareas grandes o refactorizaciones complejas
- **Reglas del Agente (agent.md):** Un archivo estándar donde defines el stack tecnológico, convenciones de estilo y arquitectura de tu proyecto. El agente lee esto siempre antes de empezar para no cometer errores de base
- **MCP (Model Context Protocol):** Un estándar para conectar la IA con herramientas externas (Notion, GitHub, bases de datos como Supabase). Esto permite que la IA, por ejemplo, lea tus tablas de base de datos directamente para saber cómo programar una consulta
- **Skills (Habilidades):** Instrucciones precisas y reutilizables (ej. "Cómo crear un componente en React según mis estándares") que el agente carga solo cuando las necesita, ahorrando memoria

## 3. El Sistema de "Memoria" y Orquestación (por Alan)

Alan presenta conceptos extremadamente avanzados para solucionar la "amnesia" de la IA:

- **Engram:** Una solución propia de Alan para dar memoria persistente a la IA entre diferentes sesiones de chat. Guarda lo aprendido sobre arquitectura, errores previos y decisiones de negocio
- **Subagentes Efímeros:** Para no saturar la memoria de la IA principal, se lanzan subagentes en nuevas ventanas de contexto para tareas específicas. Una vez terminan, devuelven solo el resultado al "padre"
- **Agent Teams Light:** Un flujo multi-agente donde varios agentes trabajan en paralelo: uno explora el código, otro propone, otro escribe la especificación, otro diseña y otro implementa

#### 4. Demostraciones en Vivo

- **Instalador de Gentleman Stack:** Alan muestra un comando que instala todo este ecosistema (Engram, agentes configurados, etc.) de forma gratuita y abierta
- **Creación de un proyecto real:** Se muestra cómo un orquestador de agentes crea un **CRUD con React, TypeScript y Tailwind** desde cero, siguiendo pasos de diseño y arquitectura profesional sin que el humano toque una sola línea de código, actuando solo como supervisor

#### 5. El Rol del "Human in the Loop"

Un mensaje central del video es que el programador ahora es como **Tony Stark** y la IA es su **Jarvis**. El humano debe conocer los fundamentos y la arquitectura para poder dirigir y "desafiar" a la IA cuando esta tome atajos o decisiones incorrectas

En resumen, la clase enseña a montar un **ejército de agentes coordinados** que utilizan memoria persistente y protocolos de contexto para desarrollar software profesional de alta calidad de forma casi autónoma, manteniendo siempre al programador como el director de la orquesta.

Resumen del tercer video:

#### 1. El Riesgo de la IA en Producción

Brais y Kiko explican que, aunque la IA nos permite generar código a una velocidad increíble, esto aumenta el riesgo de acumular **deuda técnica** y errores críticos. La

responsabilidad final del código siempre es del desarrollador, no de la IA. Por ello, es vital implementar sistemas de control antes de que el código llegue al usuario final.

## 2. Testing Automatizado Asistido por IA

El primer escudo de seguridad son los tests. Se enfatiza que:

- **Más código requiere más validación:** Los tests (unitarios e integración) son código que valida otro código.
- **Uso de IA:** Se puede usar la IA para generar los propios tests, pero el desarrollador debe verificar que realmente cubran los casos de negocio críticos.
- **Qué testear:** No hace falta testearlo todo; hay que priorizar lo **crítico** (autenticación, pagos, base de datos) y las funcionalidades principales del negocio.

## 3. Integración y Despliegue Continuo (CI/CD)

Se presentan los flujos de trabajo modernos para automatizar las subidas a producción:

- **Pull Requests (PR):** Incluso si trabajas solo, debes usar PRs como una "petición formal" para integrar código. Esto permite que se ejecuten chequeos automáticos antes de unir el código a la rama principal (main).
- **GitHub Actions:** Es la herramienta para crear flujos de trabajo (*workflows*). Se muestra cómo configurar un archivo .yaml para que, cada vez que se cree un PR, se ejecuten los tests automáticamente. Si los tests fallan, el botón de "Merge" se bloquea.
- **Reglas de Protección de Ramas:** Se enseña a blindar la rama main para que nadie pueda subir código directamente sin pasar por un proceso de revisión y aprobación.

## 4. IA en el Proceso de Revisión y Seguridad

Se introducen formas de meter la IA directamente en el flujo de GitHub:

- **GitHub Copilot para PRs:** Se muestra cómo la IA puede resumir los cambios de un pull request y sugerir mejoras de lógica directamente en los comentarios.

- **Security Review con Claude Code:** Kiko enseña cómo integrar una **GitHub Action de Anthropic** que utiliza Cloud Code para analizar el código en busca de vulnerabilidades de seguridad en cada subida. La IA detecta fallos (como credenciales expuestas) y explica cómo arreglarlos antes de desplegar.

## 5. Automatización de Versiones con "Release Please"

Para evitar la carga burocrática del mantenimiento de software, se presenta **Release Please** (una herramienta de Google):

- Genera automáticamente el número de versión (v1.1, v1.2, etc.) basándose en los mensajes de los commits.
- Crea automáticamente las **Release Notes** (notas de lanzamiento).
- **Plus de IA:** Se muestra cómo añadir un paso extra para que una IA (como GPT) redacte de forma "bonita" y profesional las notas de lo que ha cambiado en la nueva versión.

## 6. El Máster de Desarrollo con IA

La segunda mitad del video es una presentación detallada de la segunda edición del **Máster de Desarrollo con IA**. Brais explica que el objetivo es formar a desarrolladores como "orquestadores" que dominen tanto los fundamentos de la ingeniería de software como las herramientas de IA más avanzadas (Antigravity, Cursor, etc.). Destaca beneficios como el acceso vitalicio, la doble titulación universitaria y el apoyo de empresas como InfoJobs para la inserción laboral.

En conclusión, este video enseña que el desarrollo moderno no termina al escribir el código; termina cuando ese código está en producción bajo un **sistema automatizado de CI/CD** que utiliza IAs para testear, auditar la seguridad y documentar los cambios de manera profesional.